Bilkent University
Department of Computer Engineering
CS491/2 Senior Design Project
T2517
Moo-tion
Detailed Design Report

Ahmet Burak Kılıçkaya	22103833	burak.kilickaya@ug.bilkent.edu.tr
Canberk Özba	22101815	canberk.ozba@ug.bilkent.edu.tr
Emir Eroğlu	22202233	emir.eroglu@ug.bilkent.edu.tr
Mert Özkaya	22002460	mert.ozkaya@ug.bilkent.edu.tr
Muhammed Enes Şahin	22102919	enes.sahin@ug.bilkent.edu.tr

Supervisor: Doruk Öner

Course Instructors: Mert Bıçakçı, İlker Burak Kurt

13.03.2026

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfillment of the requirements of the Senior Design Project course CS491/2.

1. Introduction **5**
1.1 Purpose of the System 5
1.2 Design Goals 6
1.2.1 Reliability 6
1.2.2 Performance 6
1.2.3 Cost effectiveness and Marketability 6
1.2.4 Usability 7
1.2.5 Security and Privacy 7
1.2.6 Scalability 7
1.2.7 Modularity and Extendibility 7
1.2.8 Maintainability 7
1.2.9 Interoperability 8
1.2.10 Ethical and Regulatory Alignment 8
1.3 Definitions, Acronyms, and Abbreviations 8
1.4 Overview 9
2. Current Software Architecture **10**
2.1 Competitors 10
2.1.1 IoT Sensor Architectures 10
2.1.2 Enterprise Computer Vision Architectures 11
2.2 Alternative Solutions 11
2.2.1 Manual Observation 11
2.2.2 Single-Purpose Sensor Systems 12
2.2.3 Generic Camera Analytics 12
2.2.4 The Moo-tion Differentiator 12
3. Proposed Software Architecture **13**
3.1. Overview 13
3.2. Subsystem Decomposition 13
3.3. Persistent Data Management 16
3.4. Access Control and Security 17
3.5. Machine Learning & Image Processing Models 18
3.5.1 Detection Model (Improved YOLOv8n) 18
3.5.2 Tracking Model (TransT) 18
4. Subsystem Services **19**
4.1 Client and Mobile Application Services 19
4.2 Authentication and Account Management Services 20
4.3 Video Ingestion and Edge Vision Services 21
4.4 Backend API and Event Management Services 22
4.5 Notification and Alert Delivery Services 22
4.6 Data Storage and Media Services 23

4.7 End-to-End Service Interaction....24
5. Test Cases.... 24
5.1 Functional Test Cases.... 25
5.1.1 User Account and Authentication Test Cases....25
5.1.2 Farm and User Management Test Cases.... 32
5.1.3 Camera Management Test Cases.... 33
5.1.4 Vision Pipeline and Detection Test Cases....36
5.1.5 Notification and Alert Test Cases.... 39
5.1.6 Integration Test Cases.... 39
5.2 Non Functional Test Cases.... 43
5.2.1 Performance Test Cases....43
5.2.2 Usability Test Cases....46
5.2.3 Reliability Test Cases....47
5.2.4 Scalability Test Cases.... 49
5.2.6 Compatibility Test Cases....51
6. Considerations of Various Factors in Engineering Design.... 53
6.1. Constraints.... 53
6.1.1 Camera Access and Network Restrictions....53
6.1.2 Dependence on Existing IP Camera Infrastructure....53
6.1.3 Camera Angle and Field of View Limitations.... 54
6.1.4 Low Resolution and Image Quality Constraints.... 54
6.1.5 Illumination and Nighttime Monitoring Conditions....55
6.1.6 Barn Layout and Environmental Irregularities....55
6.1.7 Connectivity Problems in Remote Farm Locations.... 55
6.1.8 Deployment Variability Across Farms.... 56
6.1.9 Design Trade-off Between Cost Efficiency and Control.... 56
6.2. Standards.... 57
6.2.1 Video Streaming and Camera Communication Standards.... 57
6.2.2 Backend Communication and API Conventions.... 57
6.2.3 Authentication and Access Control Practices.... 58
6.2.4 Data Handling and Software Interoperability Standards.... 58
6.2.5 Modular Software Engineering Principles.... 59
6.2.6 Reliability, Maintainability, and Documentation Practices.... 59
6.2.7 Privacy and Responsible System Operation....59
7. Teamwork Details....60
7.1. Contribution.... 60
7.2. Collaborative and Inclusive Environment.... 61
8. Conclusion.... 62

9. References..... 64

1. Introduction

1.1 Purpose of the System

Reproductive inefficiency is one of the most significant operational and financial challenges in modern dairy farming. Missing a single estrus cycle may delay insemination and postpone calving by approximately one reproductive cycle, leading to extended non-productive periods and reduced herd profitability. In practice, many farms still rely on manual observation to identify estrus-related behavior, which is labor-intensive, inconsistent, and difficult to sustain continuously, especially in large herds or during night hours. Existing automated solutions attempt to reduce this burden, yet many of them depend on invasive internal sensors or maintenance-heavy wearable hardware, introducing additional cost, operational complexity, and animal handling requirements.

Moo-tion is developed to address this gap through a non-invasive, computer-vision-based decision support system for dairy cattle. The primary purpose of the system is to detect estrus-related behavior, support calving monitoring, and provide continuous herd activity observation in near real time without requiring animal-mounted or animal-ingested sensors. Instead of introducing new specialized hardware into the barn environment, Moo-tion is designed to reuse existing IP security cameras through standard RTSP streams. This makes the system passive from the animal's perspective while keeping installation burden and deployment cost low.

At a system level, Moo-tion continuously ingests video frames from barn cameras, preprocesses them, detects cattle, preserves identity across frames, and analyzes behavior patterns associated with reproductive events, such as increased activity, restlessness, and mounting-related interactions. When the system confirms a relevant event, it generates an alert and forwards it to backend services, from which it is delivered to the user through a mobile application. In this way, Moo-tion does not simply monitor video; it transforms raw visual data into actionable herd-management information that can assist farmers in making timely decisions.

Another important purpose of Moo-tion is to make precision livestock farming more accessible to small and medium-sized farms. Many existing solutions solve only one part of the problem, such as measuring body temperature or activity, while imposing hardware invasiveness, maintenance overhead, or expensive proprietary infrastructure. Moo-tion instead aims to combine affordability, non-invasiveness, and practical utility within a single software-first platform. By emphasizing compatibility with standard camera infrastructure and straightforward farmer-facing alerts, the project seeks to democratize AI-assisted reproductive monitoring in dairy farming.

1.2 Design Goals

The design of Moo-tion is shaped by both software engineering principles and the operational realities of dairy farms. The system must function under challenging visual conditions, provide timely and understandable outputs to non-technical users, remain affordable for its target market, and support growth in herd size and camera count. For these reasons, the platform is guided by the following design goals.

1.2.1 Reliability

Reliability is one of the most critical goals of Moo-tion because the system operates in visually difficult agricultural environments. Barn footage may contain dynamic lighting, shadows, fences, mud contamination, crowding, and frequent temporary occlusions between animals. Under these conditions, simple frame-level detection is not sufficient. The system must preserve identity consistency over time and continue operating robustly even when visibility conditions are imperfect. Accordingly, Moo-tion emphasizes persistent tracking, re-identification mechanisms, and low-light enhancement techniques such as CLAHE to improve reliability during both day and night operation.

1.2.2 Performance

The system must provide outputs quickly enough to remain operationally useful. Although Moo-tion is not a hard real-time control system, delayed alerts reduce the practical value of reproductive monitoring. Therefore, the overall pipeline, from behavioral event confirmation to user notification, must operate in near real time. Efficient frame processing, streamlined event generation, and responsive communication between the computer vision module, backend services, and mobile application are central to meeting this goal.

1.2.3 Cost effectiveness and Marketability

A major design objective of Moo-tion is to lower the barrier to adoption for precision livestock farming. Unlike many commercial solutions that require proprietary cameras, swallowed boluses, or wearable hardware, Moo-tion is designed to work with existing IP cameras commonly found on farms. This reduces capital expenditure, simplifies installation, and improves commercial feasibility, especially for small and medium-sized farms. The system is therefore designed not only to be technically effective, but also to be realistically deployable and marketable within its target environment.

1.2.4 Usability

The primary users of Moo-tion are farmers and herd managers rather than machine learning specialists. For this reason, the system must present complex detection outputs in a simple and understandable way. The mobile application is expected to provide readable alerts, clear event categories, intuitive navigation, and straightforward configuration features. Usability in this context means minimizing cognitive load so that users can quickly understand what happened, when it happened, and whether intervention is necessary.

1.2.5 Security and Privacy

Because Moo-tion processes live video streams collected from private agricultural facilities, security and privacy are essential design goals. The platform must ensure secure authentication, protected transmission of data, and controlled access to farm-specific alerts and visual records. Mechanisms such as JWT-based authentication and role-based access control support this objective. In addition, video data may contain not only animals but also farm workers or visitors, making responsible data access, storage, and retention policies especially important.

1.2.6 Scalability

The architecture must remain effective as the number of monitored animals, barns, and camera streams increases. Moo-tion is therefore designed with a separation between vision processing and backend services so that compute-intensive analysis can scale independently of user-facing APIs and data management modules. This allows the platform to grow from small deployments to larger farm settings without requiring a complete redesign of the system.

1.2.7 Modularity and Extendibility

Moo-tion is intentionally designed as a modular system. Detection, tracking, behavior analysis, backend logic, and mobile delivery are separated into distinct components with clear responsibilities. This improves maintainability and also allows the platform to evolve in the future. For example, the same architectural framework could later support additional monitoring capabilities such as lameness detection, general health observation, or behavior analytics for different livestock categories.

1.2.8 Maintainability

Maintainability is essential because Moo-tion combines computer vision, backend services, mobile software, and deployment in real farm conditions. The system is

designed to require minimal on-site technical intervention and to support remote updates, consistent packaging, and accessible logs for debugging and system improvement. Containerized deployment and clearly separated service boundaries help reduce long-term maintenance burden and make future updates more manageable.

1.2.9 Interoperability

Another practical design goal is interoperability with existing farm infrastructure. Moo-tion is designed to work with standard RTSP-enabled IP cameras and common video formats such as H.264 and H.265. It is also intended to support cross-platform mobile usage on both iOS and Android. This reduces vendor lock-in and aligns with the broader goal of integrating into current farm environments rather than forcing farms to adopt an entirely new hardware ecosystem.

1.2.10 Ethical and Regulatory Alignment

Moo-tion is designed as a decision-support system, not as a replacement for veterinary judgment or professional herd management expertise. Its outputs are intended to assist timely observation and intervention rather than automate critical biological decisions without human oversight. In addition, the system must respect animal welfare principles, farm data ownership, and appropriate retention and access policies. These ethical and regulatory considerations directly shape the acceptable design space of the platform.

1.3 Definitions, Acronyms, and Abbreviations

- AI: Artificial Intelligence
- API: Application Programming Interface
- Bolus: An electronic IoT sensor swallowed by cattle that rests in the reticulum to measure internal temperature and activity.
- CLAHE: Contrast Limited Adaptive Histogram Equalization. A computer vision technique used to improve visibility and contrast in low-light/nighttime video frames.
- CNN / 3D-CNN: Convolutional Neural Network. A class of deep neural networks commonly applied to analyzing visual imagery. 3D-CNNs incorporate a temporal dimension for video action recognition.

- Deep Re-ID: Deep Re-Identification. The process of extracting deep feature vectors from an object to recognize its specific identity across different frames or camera views.
- Estrus: The period of sexual receptivity in female mammals (heat), characterized in cattle by increased physical activity and standing to be mounted.
- FCM: Firebase Cloud Messaging. A cross-platform messaging solution used to reliably deliver push notifications to mobile devices.
- JWT: JSON Web Token. An open standard used to securely transmit information between parties as a JSON object, utilized for authentication.
- LSTM: Long Short-Term Memory. A recurrent neural network architecture utilized to recognize patterns over time, useful for temporal behavior tracking.
- PLF: Precision Livestock Farming
- REST: Representational State Transfer
- RTSP: Real-Time Streaming Protocol. A network control protocol designed for use in entertainment and communications systems to control streaming media servers.
- YOLO: You Only Look Once. A state-of-the-art, real-time object detection algorithm.

1.4 Overview

This report presents the detailed design of Moo-tion and explains how its software architecture addresses the practical limitations of current livestock monitoring technologies. Section 2 examines the current solution landscape, including major competitors and alternative architectural approaches used in precision livestock farming. Section 3 introduces the proposed Moo-tion architecture and its subsystem decomposition. Section 4 defines the main subsystem services and their interactions. Section 5 presents the test cases prepared to evaluate both functional correctness and non-functional system qualities. Section 6 discusses the broader engineering design constraints and standards relevant to the project. Finally, Section 7 summarizes teamwork, contribution distribution, and leadership sharing throughout development.

2. Current Software Architecture

The current precision livestock farming landscape is fragmented. Existing technologies that aim to improve reproductive monitoring and herd observation generally fall into two dominant architectural paradigms: sensor-centered monitoring systems and enterprise-grade computer vision systems. Both categories provide partial solutions, yet both introduce important trade-offs in terms of invasiveness, cost, scalability, and operational complexity. Moo-tion is designed as a direct response to these limitations by proposing a non-invasive, software-driven system that reuses existing farm infrastructure instead of replacing it.

2.1 Competitors

2.1.1 IoT Sensor Architectures

One widely adopted market approach is the use of internal or wearable IoT sensors attached to individual animals. In this architecture, the animal effectively becomes a sensing node. Devices such as internal boluses, ear tags, or smart collars collect measurements such as temperature, activity, or motion patterns and transmit them to local hubs or gateways, which then forward the aggregated data to cloud services for analysis and alert generation.

From an engineering perspective, these systems can provide strong physiological or activity-based indicators. However, they also create significant operational overhead. Hardware must be physically deployed and maintained on each animal, batteries may require replacement, devices can be lost or damaged, and system effectiveness depends on the sustained reliability of distributed sensing hardware. These issues increase maintenance burden and reduce practical convenience for many farms.

smaXtec is a representative example of the internal sensing category. It uses a swallowed bolus that remains inside the animal and measures internal biological signals. While this enables precise internal monitoring, the solution is inherently invasive and the hardware is not recoverable once deployed. This introduces animal-welfare and environmental concerns, while also limiting flexibility from a deployment perspective.

CowManager and **Allflex SenseHub** represent the wearable branch of this architecture. CowManager relies on ear-based sensing, whereas Allflex SenseHub uses collar-based monitoring to infer estrus from activity-related signals. Although these solutions are valuable, they impose recurring maintenance demands. Ear tags may be lost during herd interaction, collars require physical adjustment as animals grow, and both approaches rely on long-term management of distributed hardware units.

Consequently, their software and operational model is tightly coupled to physical device maintenance.

2.1.2 Enterprise Computer Vision Architectures

A second major category includes camera-based livestock monitoring systems that use proprietary industrial infrastructure. Compared to sensor-based solutions, these systems are non-invasive and conceptually closer to Moo-tion because they rely on visual observation instead of direct-contact sensing. However, many such systems assume enterprise-scale installation budgets and specialized infrastructure that are not practical for many dairy farms.

Cainthus is a representative example of enterprise-grade agricultural computer vision. Its strength lies in large-scale barn and feed analytics supported by advanced visual processing. However, this comes at the cost of proprietary deployment assumptions, more complex installation, and substantial infrastructure expenditure. While technically powerful, such systems are often economically inaccessible to the small and medium-sized farms that would benefit most from reproductive monitoring automation.

OneCup AI (BETSY) is among the closest conceptual competitors to Moo-tion because it also uses vision-based animal identification and monitoring. Nevertheless, its operational emphasis has been broader health verification and beef-cattle-focused workflows rather than specifically targeted estrus and calving support in dairy farming. As a result, although the underlying technology space overlaps, the problem framing and user-facing value proposition differ from Moo-tion's reproductive-monitoring focus.

Overall, the competitor landscape shows that most existing systems optimize one axis of the problem while weakening another. Sensor-based systems provide direct measurements but introduce invasiveness and maintenance burden. Enterprise-grade vision systems reduce invasiveness but often require costly proprietary infrastructure. This creates a gap for a solution that is both non-invasive and economically deployable.

2.2 Alternative Solutions

2.2.1 Manual Observation

The oldest and still common alternative is direct human observation by farm staff. This method has the advantage of requiring little technological investment, but it is inconsistent and difficult to scale. Estrus-related behaviors may be short, subtle, and easy to miss, especially in large herds or during nighttime hours. In addition, labor shortages and the gradual loss of experienced herdsmen further reduce the reliability of

manual detection. As a result, manual observation remains an important baseline practice but not a robust long-term solution.

2.2.2 Single-Purpose Sensor Systems

Another alternative is the use of single-purpose systems that monitor estrus indirectly through one type of signal, such as body temperature or general activity level. These systems may reduce some of the burden of observation, but they often provide only partial evidence rather than direct behavior-level interpretation. They also inherit many of the maintenance liabilities of wearable or internal sensing systems, including battery replacement, hardware attrition, and repeated physical handling of animals. Therefore, they address only part of the reproductive monitoring problem.

2.2.3 Generic Camera Analytics

A further alternative is the use of general-purpose camera analytics that monitor herd movement, occupancy, or feeding patterns without specifically focusing on estrus and calving behavior. While such systems may be useful for broad farm analytics, they are not optimized for the temporal and interaction-based cues required in reproductive monitoring. Detecting cattle presence in a frame is not equivalent to recognizing mounting interactions, abnormal restlessness, or specific sequences associated with estrus. For this reason, generic herd analytics systems do not fully satisfy the needs of targeted reproductive decision support.

2.2.4 The Moo-tion Differentiator

Moo-tion proposes a third path between invasive sensing and proprietary enterprise vision. It adopts the non-invasive advantages of camera-based monitoring while avoiding the high deployment cost usually associated with enterprise computer vision systems. At the same time, it avoids the maintenance-heavy and invasive nature of IoT sensor architectures by not requiring animal-mounted or animal-ingested hardware.

The central differentiator of Moo-tion is its software-first architecture. Instead of forcing farms to replace their infrastructure, the system is designed to ingest standard RTSP streams from existing IP cameras, process them through a modular AI pipeline, store relevant event data in backend services, and notify users through a mobile application. By concentrating complexity in software rather than specialized hardware, Moo-tion reduces capital expenditure, simplifies deployment, and improves long-term maintainability.

Another important differentiator is the project's targeted problem focus. Rather than attempting to solve every livestock analytics problem simultaneously, Moo-tion is specifically optimized for reproductive monitoring tasks such as estrus detection and calving-related alerts. This focused scope makes the system more relevant to immediate farm operations and improves the practical utility of its alerts.

In summary, current solutions tend to force farms into one of two trade-offs: invasive but interpretable sensor networks, or non-invasive but expensive enterprise vision installations. Moo-tion offers an alternative that is non-invasive, camera-based, software-driven, and compatible with existing infrastructure, making it both technically coherent and economically aligned with the realities of SME dairy farms.

3. Proposed Software Architecture

3.1. Overview

The Moo-tion system follows a multi-layered architecture designed to provide a scalable, secure, and real-time monitoring platform for dairy farming. The system leverages state-of-the-art computer vision and transformer-based tracking models to automate the detection of estrus behaviors, specifically targeting mounting actions.

The system architecture comprises a Client Layer, a Backend Layer, a Vision Pipeline Layer, and a Data Layer. The Client Layer utilizes the React Native framework to deliver a seamless, cross-platform mobile application for farm managers and workers. The Backend Layer, built using the FastAPI framework in Python, serves as the core of the system, managing user authentication, camera configurations, alert dispatching, and asynchronous requests. FastAPI was selected for its high performance and native asynchronous support, which is critical when interfacing with heavy machine learning inferences.

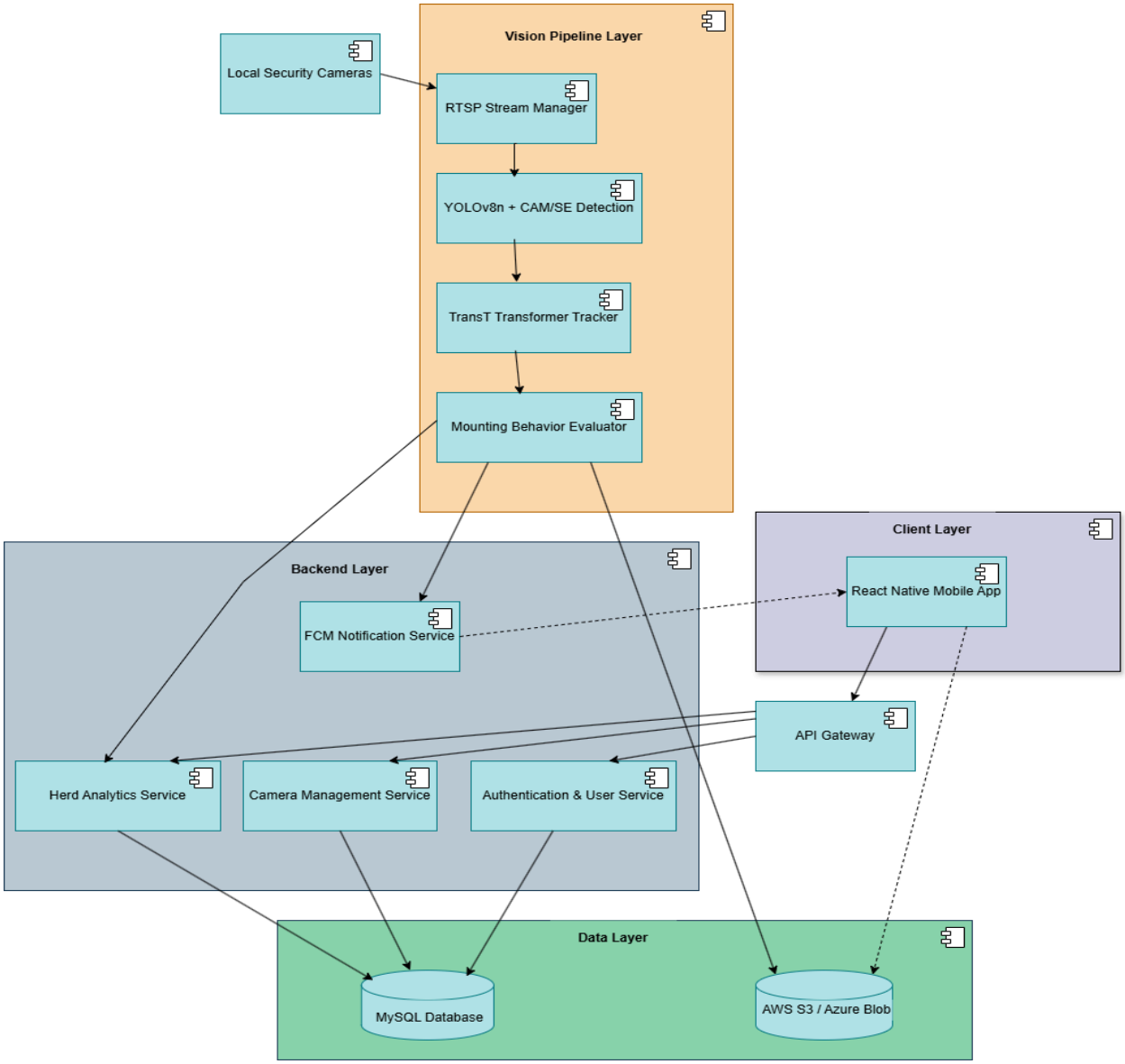
The Vision Pipeline Layer operates locally at the edge (on the farm) to minimize bandwidth consumption. It integrates the YOLO-TransT model, which fuses an improved YOLOv8n detector with a Transformer Tracking (TransT) network. Finally, the Data Layer ensures the structured storage and retrieval of farm analytics, user credentials, and event snapshots without over-complicating the infrastructure with unnecessary vector databases. This modular design enhances scalability, maintainability, and seamless user interaction.

3.2. Subsystem Decomposition

The application follows a modular microservices-based architecture, ensuring scalability, flexibility, and maintainability. The system is divided into multiple subsystems,

each responsible for a specific functionality, enabling independent development, testing, and updates.

Below is Moo-tion's subsystem decomposition diagram:



Each subsystem is documented separately below to outline its specific responsibilities within the larger platform:

- **Front-End Subsystem (Client Layer):** Responsible for the user interface and user experience, ensuring an intuitive interaction for farm workers and managers.

It provides real-time updates, seamless navigation, and a responsive design for monitoring the herd.

Responsibilities:

1. **UI Rendering:** Displaying the farm dashboard, daily activity summaries, and real-time alert histories.
2. **API Communication:** Fetches camera statuses, user settings, and historical event data from the backend services.
3. **Alert Management:** Receives and displays high-priority push notifications when estrus behavior is detected.
4. **Media Viewing:** Loads and displays snapshot images of detected events directly from cloud storage for visual verification.

- **User Authentication & Account Management Subsystem:** Responsible for secure user authentication, session management, and profile customization while ensuring data privacy. It allows users to log in securely and restricts unauthorized access to farm data.

  Responsibilities:

  1. **User Authentication:** Supports secure login using email and password credentials.
  2. **Session Management:** Maintains persistent user sessions by utilizing JWT (JSON Web Token) authentication.
  3. **Farm Authorization:** Ensures that a user can only access the camera streams and alerts associated with their explicitly assigned Farm ID.

- **Vision & Tracking Subsystem (Edge Layer):** This is the AI-driven subsystem responsible for ingesting video, detecting cattle, and tracking estrus behaviors in real-time. Operating locally on the farm, this subsystem handles the heavy computational lifting.

  Responsibilities:

  1. **Stream Ingestion:** Captures live RTSP video feeds from the farm's local IP cameras.

2. **Target Detection:** Utilizes the improved YOLOv8n model to detect cattle and specific postures indicative of estrus.

3. **Continuous Tracking:** Employs the TransT (Transformer Tracking) model to lock onto a detected cow and track its trajectory across subsequent frames, even during brief occlusions or complex herd movements.

4. **Behavior Evaluation:** Analyzes the tracking data to confirm if the mounting behavior meets the required duration threshold to trigger an estrus warning.

- **Backend Services Subsystem:** Serves as the central communication hub, processing tracking data from the edge pipeline and serving data to the mobile clients.

  Responsibilities:

  1. **Event Routing:** Accepts requests from the Vision Pipeline containing detected events and logs them into the database.

  2. **Notification Dispatching:** Communicates with Firebase Cloud Messaging (FCM) to push critical alerts to the specific devices registered to a farm.

- **Database & Persistent Data Management Subsystem:** Responsible for securely storing and managing all essential user, camera, and event data.

  Responsibilities:

  1. **Structured Data Storage:** Maintains user profiles, farm configurations, and the textual metadata of all detected estrus events.

  2. **Media Storage:** Securely stores image snapshots of mounting behaviors, ensuring fast retrieval without bloating the relational database.

3.3. Persistent Data Management

Moo-tion relies on a MySQL database as the primary data store to handle user information, camera configurations, and event logs efficiently. MySQL provides an optimal balance of read/write speeds and data integrity, supported by robust relational mapping using SQLAlchemy as the ORM (Object Relational Mapper). This ensures non-repetitive, readable, and maintainable queries within the FastAPI backend.

The database architecture is designed with the following key entities:

- **Users:** Stores user credentials (hashed passwords), email addresses, roles (e.g., Administrator, Farm Worker), and FCM device tokens for notifications.

- **Farms & Cameras:** Stores the organizational structure of the farm and the connection details (RTSP URLs, physical locations) for the camera hardware.

- **Event Logs:** Stores the metadata of detected behaviors, including the timestamp, the specific camera ID that captured the event, the behavior classification (e.g., "Mounting"), and a reference URL to the event snapshot.

Since storing binary image data directly in a relational database leads to severe performance degradation over time, Moo-tion utilizes AWS S3 (or Azure Blob Storage) for media asset management. When the YOLO-TransT pipeline detects an estrus event, a cropped JPEG image of the behavior is uploaded to the cloud storage bucket. The resulting secure URL is saved in the MySQL Event Logs table. The mobile client subsequently fetches the image using this reference URL, ensuring fast content delivery.

3.4. Access Control and Security

Given that Moo-tion processes live video feeds of private agricultural facilities, privacy and security are strictly regulated. The application enforces a multi-layered security approach to safeguard user data and prevent unauthorized access.

All public endpoints accessed by the mobile client are secured with JWT (JSON Web Tokens). When a user successfully authenticates using their email and securely hashed password (via bcrypt), the Authentication Service issues a token. Requests that lack a valid token are immediately rejected. Furthermore, Role-Based Access Control (RBAC) is enforced. Farm Workers are granted read-only access to alerts, while Farm Managers possess the necessary permissions to add, edit, or delete camera configurations and user accounts.

To secure interservice communication, particularly between the on-premise Edge Vision Pipeline and the Cloud Backend, API keys are utilized. The Edge Node authenticates its event data payloads using a secret key assigned to that specific farm, preventing arbitrary or malicious data injection into the backend. Additionally, all network traffic is encrypted in transit using SSL/TLS (HTTPS), ensuring that data cannot be intercepted. Finally, the RTSP video streams themselves remain confined to the farm's internal network, ensuring maximum privacy for the facility.

3.5. Machine Learning & Image Processing Models

The core innovation of the Moo-tion system is its highly specialized machine learning pipeline, designed specifically to detect the brief and subtle indicators of bovine estrus, predominantly mounting behavior. The system employs the YOLO-TransT architecture, which effectively integrates target detection with ongoing tracking capabilities.

3.5.1 Detection Model (Improved YOLOv8n)

To achieve accurate detection of estrus cows in complex, crowded natural scenes, the system relies on an improved version of the YOLOv8n (Nano) model. While standard YOLO models are highly efficient, the specific task of identifying mating and mounting behaviors requires a deeper contextual understanding of the scene.

To address this, two critical modifications are integrated into the YOLOv8n network:

1. **Context Augmentation Module (CAM):** This module is introduced to capture scene-related information. Rather than just identifying the shape of a cow, CAM helps the network understand the spatial correlation between two overlapping cattle, which is the defining characteristic of mounting behavior.

2. **Squeeze-and-Excitation (SE) Module:** A channel attention mechanism is integrated to discard redundant background features (such as barn walls or feeding troughs) and amplify the specific visual features associated with the cows in heat.

These improvements allow the detection model to achieve exceptionally high accuracy (AP) for estrus behavior while keeping the parameters and FLOPs lightweight enough for real-time edge processing.

3.5.2 Tracking Model (TransT)

Once an estrus cow is detected by the YOLOv8n module, the system must follow the individual to confirm the duration and validity of the behavior. For this, the system employs the Transformer Tracking (TransT) model.

Unlike traditional correlation-filter-based trackers that struggle with deformation, TransT utilizes a self-attention and cross-attention mechanism inherent to Transformer architectures. When the detection module passes a bounding box to TransT, the tracker extracts a template feature and dynamically correlates it with the search region in subsequent video frames. This allows the system to maintain a robust tracking success

rate even when the cow undergoes severe deformation (e.g., changing posture during mounting) or temporary occlusion by other members of the herd.

By fusing the improved YOLOv8n detection with the TransT tracker, Moo-tion accurately locates and follows estrus individuals, discarding the need for heavy, persistent vector databases and providing a highly efficient, end-to-end warning system.

4. Subsystem Services

This section defines the main services provided by the Moo-tion subsystems and explains how they interact to support end-to-end estrus monitoring. While Section 3 focuses on the structural decomposition of the architecture, this section describes the concrete services exposed by each subsystem from an operational perspective. In Moo-tion, the complete monitoring workflow begins with live RTSP video ingestion, continues with edge-side computer vision and behavior analysis, and ends with backend logging, alert generation, and mobile delivery. This service-oriented view is important because the platform is designed not as a single monolithic application, but as a coordinated set of components that exchange data and responsibilities through clearly separated interfaces.

4.1 Client and Mobile Application Services

The Client Layer provides the farmer-facing services of the platform. These services are responsible for presenting alerts, enabling monitoring actions, and allowing users to interact with the system through a mobile application. Since the primary users of Moo-tion are farm managers and workers rather than technical personnel, the client services are designed to expose system outputs in a direct and interpretable manner.

Dashboard and Monitoring Service:

This service presents the current system state to the user through a central dashboard. It displays recent alerts, daily activity summaries, and the general monitoring status of the connected farm. By aggregating event information into a single view, this service reduces the need for users to interpret low-level system outputs manually.

Live Stream Viewing Service:

This service allows the user to access live camera feeds associated with their farm. It acts as the visual verification layer of the system, enabling the farmer to inspect the monitored environment directly when an alert is received or when general monitoring is required.

Alert Browsing and Detail Inspection Service:

This service enables users to browse active and historical estrus alerts and inspect the related snapshots or metadata. Each alert record is expected to include information such as event time, camera source, and behavioral classification. A more detailed alert screen may also expose the corresponding image thumbnail or clip reference for verification purposes.

Settings and Configuration Service:

The client provides interfaces for adjusting notification preferences and, depending on role permissions, configuring basic operational settings such as camera sources or thresholds. This service ensures that system interaction remains user-driven while keeping the interface simple enough for field conditions.

These client-side services collectively form the operational interface of Moo-tion. Their role is not to perform analysis, but to expose backend-managed information in a form that supports timely farm decisions.

4.2 Authentication and Account Management Services

Authentication and account management services ensure that only authorized users can access farm-specific data and monitoring functionality. Because Moo-tion processes live video streams and private event records from real farms, these services are essential for safe operation.

Authentication Service:

This service manages user login and credential verification. Users authenticate with email and password credentials, and successful authentication results in the issuance of a secure session token. Requests made without a valid token are rejected by the backend.

Session Management Service:

This service maintains persistent user sessions through JWT-based authentication. It ensures that the mobile application can securely access protected endpoints without requiring the user to log in repeatedly for each action.

Farm Authorization Service:

This service enforces farm-level isolation by ensuring that users can only view the cameras, alerts, and settings associated with their assigned farm. It prevents cross-farm data exposure and acts as an important boundary in multi-farm deployments.

Role-Based Access Service:

This service differentiates capabilities between user roles such as farm workers and farm managers. For example, standard users may only view alerts and monitoring results, while users with elevated permissions may configure cameras, modify settings, or manage other accounts.

4.3 Video Ingestion and Edge Vision Services

The edge-side vision subsystem provides the core technical services of Moo-tion. These services are responsible for receiving camera streams, preparing visual data for analysis, detecting cattle, tracking behavior over time, and producing candidate estrus events. The decision to place these services near the source of the video stream is aligned with the project's low-bandwidth and low-cost deployment goals.

Stream Ingestion Service:

This service connects to one or more RTSP-enabled IP cameras and continuously receives video data from the barn environment. It is responsible for stable stream acquisition and acts as the entry point of the computer vision pipeline.

Frame Decoding and Preprocessing Service:

After stream ingestion, incoming video is decoded and prepared for analysis. This service performs resolution scaling, frame sampling, and other preprocessing steps required for efficient inference. In low-light scenarios, image enhancement operations may also be applied to improve the quality of nighttime analysis.

Object Detection Service:

This service uses the improved YOLOv8n model to detect and localize cattle in each frame. Its role is to convert raw images into structured detections that can be passed to downstream tracking and behavior analysis stages.

Tracking and Identity Continuity Service:

Once cattle are detected, the tracking service uses the TransT model to preserve identity across frames and maintain continuous trajectories. This is essential because estrus-related behavior is inherently temporal and cannot be reliably determined from isolated frames alone.

Behavior Evaluation Service:

This service analyzes tracked movement patterns and mounting-related interactions in order to decide whether the observed sequence corresponds to a meaningful estrus event. It applies temporal conditions and behavioral thresholds so that alerts are generated only when sufficient evidence is accumulated, reducing false positives from short or ambiguous movements.

Together, these services form the edge-side analytical core of Moo-tion and are responsible for transforming video frames into structured event candidates.

4.4 Backend API and Event Management Services

The Backend Layer acts as the central communication and coordination hub of the platform. It receives event data from the edge pipeline, stores and organizes records, and serves mobile clients through authenticated APIs.

REST API Service:

This service exposes backend functionality to the mobile application through authenticated HTTP endpoints. It supports common application actions such as login, alert retrieval, camera listing, settings access, and event history queries.

Event Routing and Registration Service:

This service receives candidate estrus events from the edge vision pipeline and converts them into persistent backend records. Its role is to normalize incoming event payloads, validate them, and register them in the system with the associated metadata.

Alert Record Management Service:

This service manages the lifecycle of stored alerts. It supports retrieval of recent and historical event data, filtering by time or source, and preparation of alert information for display in the mobile client.

Camera Management Service:

This service stores and serves camera-related metadata such as RTSP addresses, logical locations, and farm associations. It enables the rest of the platform to treat cameras as manageable system entities rather than fixed hardcoded inputs.

Reporting and Export Support Service:

Although more lightweight than the real-time services, the backend also supports historical review and report-oriented retrieval of alert data. This allows users to browse prior estrus events and, where needed, export or summarize them for management purposes.

4.5 Notification and Alert Delivery Services

Notification services are responsible for converting a confirmed backend event into a timely message for the farmer. Since estrus detection is operationally useful only when delivered in a practical time window, these services are a critical part of the system.

Alert Threshold Monitoring Service:

This service monitors incoming event records and determines whether the detected behavior satisfies the conditions required for user notification. It acts as the final control layer between raw event generation and external alert delivery.

Push Notification Dispatch Service:

When a valid event is confirmed, this service communicates with Firebase Cloud Messaging (FCM) to deliver push notifications to the registered mobile devices associated with the farm. It ensures that alerts are routed only to authorized users.

Notification Formatting Service:

This service prepares the content of the outgoing notification, including the event category, time, and, where applicable, a visual reference such as a snapshot. Its purpose is to make alerts informative enough to support immediate farmer attention.

Acknowledgment-Oriented Delivery Support:

Although the system is primarily designed for notification delivery rather than complex workflow acknowledgment, the service structure is intended to support future alert-state extensions such as viewed, dismissed, or escalated event handling. This makes the notification subsystem extendable without changing the core monitoring logic.

4.6 Data Storage and Media Services

Persistent data services ensure that Moo-tion can store user information, camera configurations, event metadata, and visual evidence in a structured and retrievable way. The current design uses MySQL for structured records and external object storage for event media.

Structured Data Storage Service:

This service stores relational data such as users, roles, farm assignments, camera information, and textual event metadata. It provides the persistent backbone for authentication, event history, and application configuration.

Event Logging Service:

Each confirmed event is logged with metadata such as timestamp, camera ID, event type, and reference information. This service ensures traceability and enables later review of detections through the mobile application or reporting functions.

Media Storage Service:

Instead of storing image binaries directly in the relational database, Moo-tion uses cloud object storage such as AWS S3 or Azure Blob Storage for event snapshots.

This design improves performance and avoids database bloat while still allowing rapid retrieval of visual evidence.

Snapshot Retrieval Service:

This service allows the mobile application to fetch stored event images through secure references saved in the structured database. In practice, this gives the user a lightweight but useful way to visually verify the circumstances of an alert.

These services collectively support both the real-time and retrospective aspects of the system by ensuring that important monitoring outputs remain securely stored and accessible when needed.

4.7 End-to-End Service Interaction

The subsystem services of Moo-tion are designed to operate in a coordinated end-to-end workflow. First, the video ingestion and preprocessing services receive the RTSP stream from the existing farm camera. Second, the edge vision services detect cattle, track them over time, and evaluate whether the observed behavior is indicative of estrus. Third, confirmed events are forwarded to the backend, where they are validated, stored, and linked with their metadata and media references. Fourth, the notification subsystem triggers push alerts for the relevant users. Finally, the mobile application services allow the farmer to inspect live streams, browse alert history, and review the visual evidence associated with the event. This service chain ensures that Moo-tion functions as a coherent decision-support platform rather than a disconnected collection of technical modules.

5. Test Cases

This section outlines the test cases that will be executed to validate the functionality, integration, and reliability of the Moo-tion system during and after implementation. The test cases are designed to ensure that all components of the system—including the mobile application, backend services, vision pipeline, and data storage—operate correctly and interact seamlessly.

The tests are divided into functional and non-functional test cases, each addressing different aspects of system validation. Functional tests verify that the system performs its intended operations such as camera stream ingestion, cattle detection,

estrus behavior identification, event logging, and notification delivery. Non-functional tests evaluate system qualities including performance, security, usability, and scalability.

Each test case is also categorized according to its severity level, which indicates the importance of the tested functionality. Critical severity represents essential system operations that must function correctly for the platform to operate. Major severity corresponds to features that significantly affect system usability or monitoring effectiveness but may have temporary workarounds. Minor severity covers features that do not directly impact the core monitoring pipeline but still contribute to overall system quality and usability.

By executing these tests, we aim to ensure that the Moo-tion platform delivers a reliable and efficient monitoring solution for dairy farms. The testing process verifies that the system can accurately detect estrus behaviors, process and store event data, and notify farm personnel in a timely and secure manner while maintaining system stability and usability.

5.1 Functional Test Cases

Functional test cases focus on verifying whether the main features of the Moo-tion system operate as expected. These tests validate the behavior of the system's primary modules and ensure that data flows correctly between different subsystems.

The functional tests are organized into several categories including user authentication and authorization, camera and farm configuration management, vision pipeline processing, event detection and tracking, backend event processing, notification delivery, and mobile application interaction.

These tests ensure that users can securely access the system, configure monitoring cameras, receive alerts when estrus behavior is detected, and review historical monitoring data through the mobile interface. Additionally, they verify that the computer vision pipeline can reliably detect cattle, track individuals across frames, and correctly identify mounting behavior events without generating false alerts.

Through these tests, we ensure that the Moo-tion system integrates its multiple components—including RTSP camera streams, machine learning models, backend services, and mobile interfaces—into a cohesive and reliable precision livestock monitoring platform.

5.1.1 User Account and Authentication Test Cases

Test ID	FR-AUTH-001A	Category	Security
Objective	To verify that a user can successfully log in with valid credentials.		
Steps	1. Open the mobile application. 2. Navigate to the Login page 3. Enter a valid email address and password 4. Click the "Login" button.		
Expected	The system should authenticate the user and allow access to the dashboard.		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-001B	Category	Security
Objective	To verify that the system rejects login attempts with invalid credentials.		
Steps	1. Open the mobile application 2. Navigate to the Login page. 3. Enter an invalid email or password. 4. Click the "Login" button.		
Expected	The system should deny access and display an authentication error message.		

Date-Result	15.11.2025 / Success

Test ID	FR-AUTH-002	Category	Security
Objective	To verify that protected system resources require authentication.		
Steps	1. Open the application. 2. Attempt to the access the dashboard without logging in.		
Expected	The system should redirect the user to the login page.		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-003	Category	Security
Objective	To verify that role-based access control restricts unauthorized actions.		
Steps	1. Login as a Farm Worker. 2. Attempt to modify the camera settings.		
Expected	The Farm Workers should not be allowed to change the settings.		
Date-Result	TBD		

Test ID	FR-AUTH-004	Category	Security
Objective	To verify that role-based access control restricts unauthorized actions.		
Steps	1. Login as a Farm Manager. 2. Attempt to modify the camera settings.		
Expected	The system successfully changed the camera settings.		
Date-Result	TBD		

Test ID	FR-AUTH-005	Category	Security
Objective	To verify that the system invalidates the user session after logout.		
Steps	1. Log in successfully to the mobile application. 2. Navigate to the settings or account menu. 3. Click the "Logout" button. 4. Attempt to access the dashboard using the previous session.		
Expected	The system should terminate the session and redirect the user to the login page.		
Date-Result	TBD		

Test ID	FR-AUTH-006	Category	Security
Objective	To verify that a Farm Worker cannot create, edit, or delete user accounts.		
Steps	1. Log in as a Farm Worker. 2. Navigate to any available user or account management interface. 3. Attempt to create a new user. 4. Attempt to edit an existing user. 5. Attempt to delete an existing user.		
Expected	The system should deny all account management actions for the Farm Worker role.		
Date-Result	TBD		

Test ID	FR-AUTH-007	Category	Security
Objective	To verify that the system rejects API requests made with an expired authentication token.		
Steps	1. Log in to obtain a valid authentication token. 2. Wait until the token expires or manually simulate expiration. 3. Send a request to a protected API endpoint using the expired token.		

Expected	The system should reject the request and return an authentication error.
Date-Result	TBD

Test ID	FR-AUTH-008	Category	Security
Objective	To verify that a new user can successfully create an account using the signup process.		
Steps	1. Open the mobile application. 2. Navigate to the Signup page. 3. Enter valid name, email, and password. 4. Click the "Sign Up" button.		
Expected	The system should create a new user account and redirect the user to the email verification page.		
Date-Result	15.11.2025 / Success		

Test ID	FR-AUTH-009	Category	Security
Objective	To verify that email verification succeeds with a valid verification code.		

Steps	1. Complete the signup process. 2. Receive the verification code by email. 3. Enter the correct verification code in the verification screen. 4. Submit the code.
Expected	The system should verify the user account and allow access to the farm setup or dashboard.
Date-Result	15.11.2025 / Success

Test ID	FR-AUTH-010	Category	Security
Objective	To verify that the system rejects incorrect email verification codes.		
Steps	1. Complete the signup process. 2. Enter an incorrect verification code in the verification screen. 3. Submit the code.		
Expected	The system should display an error message and prevent account activation.		
Date-Result	15.11.2025 / Success		

5.1.2 Farm and User Management Test Cases

Test ID	FR-FARM-001	Category	Functional
Objective	To verify that a user can create a farm profile after account registration.		
Steps	1. Log in to the application. 2. Navigate to the farm creation page. 3. Enter farm name, location, and herd size. 4. Save the configuration.		
Expected	The system should create the farm profile and associate it with the user account.		
Date-Result	15.11.2025 / Success		

Test ID	FR-FARM-002	Category	Functional
Objective	To verify that a Farm Owner can invite a new staff member to the farm system..		
Steps	1. Log in as Farm Owner. 2. Navigate to the Manage Users page. 3. Click "Invite User". 4. Enter the staff member’s email and assign a role.		

	5. Send the invitation.
Expected	The system should send an invitation email and create a pending user account.
Date-Result	15.11.2025 / Success

Test ID	FR-FARM-003	Category	Functional
Objective	To verify that invited staff members can activate their account through an invitation link.		
Steps	1. Receive invitation email. 2. Click the invitation link. 3. Set a password. 4. Log in to the system.		
Expected	The system should activate the user account with the assigned role permissions.		
Date-Result	15.11.2025 / Success		

5.1.3 Camera Management Test Cases

Test ID	FR-CAM-001	Category	Functional

Objective	To verify that the system rejects incorrect email verification codes.
Steps	5. Complete signup. 6. Enter a correct verification code. 7. Submit the code.
Expected	The system should register the camera and display it in the camera list.
Date-Result	15.11.2025 / Success

Test ID	FR-CAM-002	Category	Functional
Objective	To verify that the system handles invalid camera stream configurations.		
Steps	1. Log in as a Farm Manager. 2. Navigate to the camera configuration page. 3. Enter an invalid RTSP URL. 4. Attempt to save the configuration.		
Expected	The system should reject the configuration and display an error message.		
Date-Result	TBD		

Test ID	FR-CAM-003	Category	Functional
Objective	To verify that a Farm Manager can edit an existing camera configuration.		
Steps	1. Log in as a Farm Manager. 2. Navigate to the Camera Management page. 3. Select an existing camera from the list. 4. Modify the camera location or RTSP URL. 5. Save the updated configuration.		
Expected	The system should update the camera configuration and reflect the changes in the camera list.		
Date-Result	TBD		

Test ID	FR-CAM-004	Category	Functional
Objective	To verify that a Farm Manager can delete a camera configuration.		
Steps	1. Log in as a Farm Manager. 2. Navigate to the Camera Management page. 3. Select an existing camera. 4. Click the "Delete" option.		

	5. Confirm the deletion.
Expected	The system should remove the camera from the camera list and stop processing its stream.
Date-Result	15.11.2025 / Success

Test ID	FR-Cam-005	Category	Functional
Objective	To verify that the system successfully ingests RTSP camera streams.		
Steps	1. Configure a valid RTSP camera. 2. Start the vision pipeline. 3. Monitor the incoming video stream.		
Expected	The system should successfully receive and process frames from the RTSP stream.		
Date-Result	15.11.2025 / Success		

5.1.4 Vision Pipeline and Detection Test Cases

Test ID	FR-AI-001	Category	Functional
Objective	To verify that the system detects cattle in video frames.		

Steps	1. Start the vision pipeline. 2. Provide a camera feed containing cattle. 3. Observe the detection results.
Expected	The system should detect cattle in the frame using the object detection model.
Date-Result	TBD

Test ID	FR-AI-002	Category	Functional
Objective	To verify that the system tracks detected cows across frames.		
Steps	1. Start the camera stream. 2. Detect a cow in the first frame. 3. Continue processing subsequent frames.		
Expected	The system should continuously track the same cow across frames.		
Date-Result	TBD		

Test ID	FR-AI-003	Category	Functional
Objective	To verify that the system detects estrus mounting behavior.		

Steps	1. Provide a video stream containing mounting behavior. 2. Run the behavior analysis module. 3. Observe the detection results.
Expected	The system should detect mounting behavior and classify it as an estrus event.
Date-Result	TBD

Test ID	FR-AI-004	Category	Edge Case
Objective	To verify that incomplete mounting behavior does not trigger an estrus alert.		
Steps	1. Provide a video where mounting behavior lasts less than the detection threshold. 2. Run the behavior detection pipeline.		
Expected	The system should not generate an estrus alert.		
Date-Result	TBD		

Test ID	FR-AI-005	Category	Functional
Objective	To verify that the system detects estrus mounting behavior.		

Steps	4. Provide a video stream containing mounting behavior. 5. Run the behavior analysis module. 6. Observe the detection results.
Expected	The system should detect mounting behavior and classify it as an estrus event.
Date-Result	TBD

5.1.5 Notification and Alert Test Cases

Test ID	FR-NOTIF-001	Category	Integration
Objective	To verify that the system sends push notifications when an estrus event is detected.		
Steps	1. Trigger an estrus detection event. 2. Allow the backend to process the event. 3. Observe the registered mobile device.		
Expected	The system should send a push notification to the user device.		
Date-Result	TBD		

5.1.6 Integration Test Cases

Test ID	FR-INT-001	Category	Integration

Objective	To verify that a detected mounting event is processed end-to-end from the edge vision pipeline to the mobile application alert screen.
Steps	1. Configure a valid RTSP camera for a farm. 2. Start the vision pipeline connected to the configured camera. 3. Provide a video stream containing a valid mounting behavior exceeding the detection threshold. 4. Allow the detection module to identify the target event. 5. Allow the tracking and behavior evaluation modules to confirm the event. 6. Send the event payload to the backend API. 7. Allow the backend to store the event metadata and trigger a push notification. 8. Open the mobile application with a user account assigned to the same farm. 9. Navigate to the alert history screen.
Expected	The system should detect the event, store its metadata, send a notification, and display the alert in the mobile application for the correct farm user.
Date-Result	TBD

Test ID	FR-INT-002	Category	Integration
Objective	To verify that the system stores an event snapshot in cloud storage and links it correctly to the event record in the database.		

Steps	1. Start the vision pipeline for a configured farm camera. 2. Trigger a valid estrus-related event. 3. Allow the pipeline to generate a cropped JPEG snapshot. 4. Upload the snapshot to the configured cloud storage service. 5. Store the returned snapshot URL in the event log record. 6. Query the event record from the backend. 7. Open the same event from the mobile application.
Expected	The system should upload the snapshot successfully, save the snapshot URL in the event record, and display the same image when the event is opened in the mobile application.
Date-Result	TBD

Test ID	FR-INT-003	Category	Integration
Objective	To verify that an authenticated mobile client can retrieve alert history together with the corresponding media references from the backend.		
Steps	1. Log in the mobile application using valid credentials. 2. Ensure that at least one event with a snapshot exists in the system. 3. Request alert history from the backend. 4. Select one alert entry from the list. 5. Request the associated snapshot.		

Expected	The system should return the alert history for the authorized farm and provide access to the associated media reference for each alert that includes a snapshot.
Date-Result	TBD

Test ID	FR-INT-004	Category	Integration
Objective	To verify that the tracking subsystem maintains the target identity during short-term occlusion.		
Steps	1. Start the vision pipeline on a stream containing multiple cows. 2. Detect a target cow exhibiting relevant behavior. 3. Introduce a short-term occlusion where the target is partially hidden by another cow or a barn obstacle. 4. Continue processing subsequent frames.		
Expected	The system should continue tracking the same target after the brief occlusion without assigning a new identity unnecessarily.		
Date-Result	TBD		

Test ID	FR-INT-005	Category	Integration

Objective	To verify that the backend logs detected events into the database with the correct metadata.
Steps	1. Submit a valid event payload to the backend. 2. Query the event logs in the database. 3. Compare the stored record with the submitted payload
Expected	The backend should store the event timestamp, camera ID, behavior classification, farm association, and snapshot reference correctly.
Date-Result	TBD

5.2 Non Functional Test Cases

Non-functional test cases evaluate the quality attributes of the Moo-tion system rather than its feature-specific behavior. These tests verify whether the system satisfies critical operational requirements such as performance, security, usability, reliability, scalability, compatibility, and maintainability. Since Moo-tion is designed as a real-time, secure, and modular monitoring platform operating across the vision pipeline, backend, data layer, and mobile application, these tests ensure that the system remains effective and dependable under realistic operating conditions.

5.2.1 Performance Test Cases

Test ID	NFR-PERF-001	Category	Performance
Objective	To verify that critical estrus alerts are delivered to the mobile application within the required time threshold (5 seconds).		

Steps	Start the RTSP camera stream, vision pipeline, backend service, and mobile application. Provide a valid mounting behavior event that exceeds the detection threshold. Record the time when the threshold is crossed in the vision pipeline. Record the time when the push notification is received on the mobile device.
Expected	The system should deliver the alert to the mobile application within 5 seconds after the behavioral threshold is crossed.
Date-Result	TBD

Test ID	NFR-PERF-002	Category	Performance
Objective	To verify backend API response time under normal load.		
Steps	1. Start the backend server. 2. Send requests to common endpoints (login, alert history, camera list). 3. Measure response time.		
Expected	The backend should respond to requests within 500 ms under normal system load.		
Date-Result	TBD		

Test ID	NFR-PERF-003	Category	Performance
Objective	To verify real-time processing of camera streams.		
Steps	1. Start the RTSP camera stream. 2. Run the vision pipeline continuously. 3. Measure processing delay between frame capture and inference output.		
Expected	The vision pipeline should process frames with maximum latency of 1 second.		
Date-Result	TBD		

Test ID	NFR-PERF-004	Category	Performance
Objective	To verify event snapshot retrieval performance.		
Steps	1. Ensure event snapshots are stored in cloud storage. 2. Open an alert entry in the mobile application. 3. Measure the time required for the snapshot image to load.		
Expected	The snapshot image should load within 1.5 seconds on a stable network connection.		

Date-Result	TBD

5.2.2 Usability Test Cases

Test ID	NFR-USE-001	Category	Usability
Objective	To verify dashboard readability.		
Steps	1. Log in to the mobile application. 2. Open the monitoring dashboard.		
Expected	Users should be able to locate alert information within 5 seconds without external guidance.		
Date-Result	TBD		

Test ID	NFR-USE-002	Category	Usability
Objective	To verify that alerts can be easily interpreted.		
Steps	1. Generate multiple alerts in the system. 2. View the alert history page.		
Expected	Users should correctly interpret alert details such as camera location and event time within 5 seconds.		
Date-Result	TBD		

5.2.3 Reliability Test Cases

Test ID	NFR-REL-001	Category	Reliability
Objective	To verify continuous system operation.		
Steps	1. Run the full system pipeline (camera, vision, backend). 2. Operate the system continuously.		
Expected	The system should operate continuously for at least 72 hours without critical service failure.		
Date-Result	TBD		

Test ID	NFR-REL-002	Category	Reliability
Objective	To verify system recovery from camera interruption.		
Steps	1. Start the system with a valid RTSP stream. 2. Disconnect the camera network for 30 seconds. 3. Restore the connection.		
Expected	The system should resume normal stream processing within 10 seconds after reconnection.		

Date-Result	TBD

Test ID	NFR-REL-003	Category	Reliability
Objective	To verify that stored data remains consistent after an unexpected shutdown or power outage.		
Steps	1. Start the full system and process active detections and alerts. 2. During operation, simulate an unexpected shutdown or power loss. 3. Restart the backend, database, and vision services. 4. Query the latest stored event, alert, and tracking records.		
Expected	The system should restart successfully, and previously stored data should remain consistent without corruption, duplication, or missing committed records.		
Date-Result	TBD		

Test ID	NFR-REL-004	Category	Reliability
Objective	To verify that video processing tolerates temporary network drops or camera signal loss without losing long-term tracking continuity.		
Steps	1. Start a valid camera stream and tracking pipeline. 2. Allow the system to track one or more cows over time.		

	3. Introduce a short network interruption or temporary signal loss. 4. Restore the connection and continue processing.
Expected	The system should recover after the temporary interruption and continue operation without losing long-term tracking data unnecessarily.
Date-Result	TBD

5.2.4 Scalability Test Cases

Test ID	NFR-SCAL-001	Category	Scalability
Objective	To verify support for multiple camera streams.		
Steps	1. Configure multiple RTSP cameras. 2. Run the vision pipeline.		
Expected	The system should support at least 5 simultaneous camera streams without critical performance degradation.		
Date-Result	TBD		

Test ID	NFR-SCAL-002	Category	Scalability

Objective	To verify system performance under increased event volume.
Steps	1. Simulate or submit multiple event detections rapidly. 2. Monitor backend processing and storage.
Expected	The backend should process at least 100 event submissions per minute without data loss.
Date-Result	TBD

Test ID	NFR-SCAL-003	Category	Scalability
Objective	To verify database performance with large event history.		
Steps	1. Populate the database with 10,000 event records. 2. Retrieve alert history through the mobile application.		
Expected	The system should retrieve and display event records within 3 seconds.		
Date-Result	TBD		

Test ID	NFR-SCAL-004	Category	Scalability
Objective	To verify that the system scales to farms with at least 50 cows and 4		

	simultaneous camera streams.
Steps	1. Configure the system for a farm scenario containing at least 50 cows. 2. Connect 4 simultaneous camera streams. 3. Run the vision pipeline, backend processing, and mobile alert flow under this load. 4. Monitor throughput, responsiveness, and stability.
Expected	The system should continue operating correctly at the target scale without critical failures or unacceptable degradation in core functions.
Date-Result	TBD

5.2.6 Compatibility Test Cases

Test ID	NFR-COMP-001	Category	Compatibility
Objective	To verify that the mobile application operates correctly on supported Android devices.		
Steps	1. Install the mobile application on a supported Android device. 2. Log in using valid credentials. 3. Open the dashboard, alert history, and alert detail pages. 4. Open an event snapshot image.		

Expected	The system should function correctly on supported Android devices without major interface or functional inconsistencies.
Date-Result	15.11.2025 / Success

Test ID	NFR-COMP-002	Category	Compatibility
Objective	To verify that the mobile application operates correctly on supported iOS devices.		
Steps	1. Install the mobile application on a supported iOS device. 2. Log in using valid credentials. 3. Open the dashboard, alert history, and alert detail pages. 4. Open an event snapshot image.		
Expected	The system should function correctly on supported iOS devices without major interface or functional inconsistencies.		
Date-Result	15.11.2025 / Success		

6. Considerations of Various Factors in Engineering Design

6.1. Constraints

The design and implementation of Moo-tion are shaped by a number of technical, environmental, and operational constraints that arise directly from its intended deployment in real farm environments. Unlike systems developed for fully controlled laboratory settings, Moo-tion is expected to function under practical field conditions where infrastructure quality, camera placement, lighting, and connectivity cannot always be optimized by the development team. Since the project deliberately aims to provide a low-cost and scalable monitoring solution by building on existing farm infrastructure, these real-world limitations form an essential part of the system context. For this reason, the following constraints must be considered as fundamental factors affecting both system design and expected performance.

6.1.1 Camera Access and Network Restrictions

One of the primary constraints encountered in the development of Moo-tion was limited access to the camera systems already installed in farms. Although the proposed system is designed to operate on existing IP camera infrastructure, practical deployment revealed that access to live video streams is not always straightforward. In several cases, farms used network security mechanisms such as firewalls and restricted local network policies, which prevented direct access to RTSP streams or remote camera interfaces. This created difficulties in collecting real-world video data, testing the system under realistic conditions, and validating the end-to-end operation of the platform.

This issue also highlighted an important dependency of the project: the performance and deployability of the system are partially influenced by the configuration and openness of third-party camera infrastructure. Since camera ownership and control remain on the farm side, integration may require additional technical intervention, permissions, and network-level adjustments before the software system can function reliably.

6.1.2 Dependence on Existing IP Camera Infrastructure

A major design decision of Moo-tion is to use existing IP cameras rather than introducing a dedicated sensing setup. While this approach reduces installation cost and improves scalability, it also creates a significant

constraint for the system. Existing camera infrastructures were generally not installed for intelligent livestock monitoring purposes, but rather for basic surveillance or security. As a result, many of these cameras are not optimally positioned, configured, or maintained for computer vision based behavioral analysis.

This dependence limits the level of control that can be exercised over critical visual parameters such as camera height, field of view, lens quality, frame rate, compression settings, and stream stability. Consequently, the proposed system must operate under non-ideal and variable imaging conditions, which can reduce robustness and make deployment quality inconsistent across different farms.

6.1.3 Camera Angle and Field of View Limitations

Another important constraint is related to the viewing geometry of farm cameras. In multiple cases, camera angles were not carefully selected for tracking and observing cows over time. Cameras positioned for general barn surveillance may fail to capture animals from perspectives that are suitable for reliable detection, posture interpretation, or movement analysis. Poor viewing angles may cause frequent occlusions, partial visibility, overlapping animals, or blind zones within the monitored area.

Such limitations directly affect the amount of meaningful visual information available to the system. If cows are not clearly visible or remain outside the effective field of view for long periods, monitoring continuity is weakened and behavioral interpretation becomes less reliable. Therefore, the success of the system is strongly influenced by whether the installed cameras provide an observation geometry compatible with animal monitoring tasks.

6.1.4 Low Resolution and Image Quality Constraints

The quality of the available video streams constitutes another major technical limitation. Some of the cameras accessible during the project provided low-resolution outputs, which restricted the visibility of fine visual details required for accurate computer vision analysis. Low image quality can reduce the distinguishability of animal boundaries, body posture, motion patterns, and interactions between animals and their surroundings.

In addition to resolution, image compression artifacts, motion blur, noise, and inconsistent frame quality may further degrade system performance.

Since Moo-tion relies on extracting meaningful information from visual data, insufficient image quality can reduce detection accuracy, tracking stability, and confidence in behavioral inference. This makes video quality one of the most important limiting factors in real-world deployment.

6.1.5 Illumination and Nighttime Monitoring Conditions

Lighting conditions inside barns represent a further environmental constraint. Daytime operation may be acceptable under natural or partially supported lighting conditions; however, nighttime monitoring is expected to be significantly more challenging. In many barns, artificial illumination may be weak, uneven, or poorly distributed, which can reduce contrast and overall image clarity. Dark regions, shadows, and localized glare may degrade visual consistency across the scene.

These limitations are particularly important because Moo-tion is intended to support continuous monitoring. If image quality decreases substantially at night, the system may experience reduced reliability exactly during periods when continuous observation remains operationally valuable. For this reason, insufficient nighttime lighting must be considered a deployment constraint that can directly influence the accuracy and robustness of the visual analysis pipeline.

6.1.6 Barn Layout and Environmental Irregularities

The physical structure of barns also imposes constraints on the system. Real farm environments are rarely organized in a visually clean or standardized way. Barn interiors may include irregular layouts, feeding structures, fences, machinery, shadows, cluttered backgrounds, and various static or moving objects that complicate image interpretation. These elements increase scene complexity and can create occlusions or ambiguities in the visual field.

Moreover, animals may move in groups, remain partially hidden, or overlap with environmental structures, making it difficult to isolate individual cows consistently. This irregularity reduces the predictability of the scene and makes algorithmic generalization more difficult compared with controlled laboratory conditions. As a result, the system must be designed to tolerate environmental inconsistency, but such inconsistency remains a practical constraint on performance.

6.1.7 Connectivity Problems in Remote Farm Locations

Network reliability is another important operational constraint, especially for farms located in rural or mountainous regions. In such areas, internet access and local network quality may be unstable, intermittent, or insufficient for continuous video streaming and timely communication between system components. Because Moo-tion involves edge-side video processing as well as communication with backend and mobile interfaces, degraded connectivity may interrupt stream acquisition, delay notifications, or reduce synchronization between system layers.

This is particularly relevant for real-time or near-real-time monitoring scenarios. Even if the computer vision pipeline performs correctly, unreliable communication infrastructure may limit the practical usability of the overall solution. Therefore, deployment feasibility is not determined solely by the algorithmic capabilities of the system, but also by the communication quality available at the farm site.

6.1.8 Deployment Variability Across Farms

An additional constraint stems from the heterogeneity of farm conditions. Different farms may use different camera brands, network architectures, enclosure layouts, and illumination conditions. This variability means that a system configuration that performs well in one farm may not immediately transfer to another without adjustment. In other words, deployment conditions are not standardized, and this affects the reproducibility of performance across sites.

This variability imposes additional effort in integration, calibration, and system adaptation. It also limits the assumption of uniform operating conditions during development and testing. Consequently, the system must be robust to cross-site variation, but this requirement itself reflects a major practical constraint of the project.

6.1.9 Design Trade-off Between Cost Efficiency and Control

Finally, the project involves a deliberate trade-off between affordability and environmental control. Moo-tion aims to provide a low-cost and scalable solution by using already available camera systems instead of requiring specialized hardware. However, this cost-efficient approach reduces the project team's control over the sensing environment. In practice, lower installation cost is achieved at the expense of accepting limitations in image quality, camera accessibility, and deployment consistency.

This trade-off is central to the engineering rationale of the project. The system is not intended to eliminate all environmental limitations through expensive infrastructure replacement; rather, it is designed to operate as effectively as possible within the constraints of real agricultural settings. For this reason, the identified limitations should be understood not only as obstacles, but also as realistic boundary conditions that shape the design of the proposed solution.

6.2. Standards

The Moo-tion system is designed in accordance with widely adopted software engineering, networking, and data communication standards that support interoperability, modularity, and secure operation. Although the project is developed as an academic engineering prototype rather than a formally certified industrial product, its architecture follows established technical conventions to ensure that the system remains scalable, maintainable, and compatible with existing infrastructure commonly used in livestock facilities.

6.2.1 Video Streaming and Camera Communication Standards

Since Moo-tion operates on existing farm camera infrastructure, compatibility with standard IP camera communication methods is an essential requirement. In this context, the system is designed to work with Real Time Streaming Protocol (RTSP) based camera streams, which are widely used in commercial surveillance and monitoring systems. The adoption of standard stream access methods improves integration with third-party cameras and avoids the need for proprietary hardware dependencies.

Using established video stream communication protocols also supports system flexibility across different deployment sites. Since farms may use cameras from different manufacturers, adherence to common IP camera interfacing practices is important for maintaining interoperability and reducing integration effort.

6.2.2 Backend Communication and API Conventions

Communication between the software components of Moo-tion follows common web service design conventions. The interaction between backend services, mobile application components, and other software layers is structured

according to REST-oriented API principles. Data exchanged between system components is represented in lightweight and platform-independent formats such as JSON, which simplifies integration, improves readability, and supports extensibility.

These conventions are particularly valuable in a modular system architecture, where multiple components must exchange data in a clear and consistent manner. Standardized API behavior also contributes to easier debugging, future feature expansion, and maintainability during continued development.

6.2.3 Authentication and Access Control Practices

Because Moo-tion involves access to camera streams, monitoring data, and user-facing mobile interfaces, security-oriented design practices are an important part of the system. In this context, authentication and authorization mechanisms are structured according to widely used token-based access control approaches, such as JWT-based session management where appropriate. This enables controlled access to system resources and helps prevent unauthorized use of sensitive monitoring functions.

In addition, the system design follows common secure software practices related to user credential handling, protected access paths, and controlled communication between components. While the project is not positioned as a formally certified cybersecurity product, it is developed with the intention of following accepted software security principles suitable for a real-world monitoring platform.

6.2.4 Data Handling and Software Interoperability Standards

The system is also guided by general standards of structured data handling and software interoperability. Since Moo-tion combines edge-side video analysis, backend processing, database interaction, and mobile application communication, it is necessary for data exchange mechanisms to remain consistent across all layers. Standardized data formatting, timestamp handling, and service communication structures are therefore important design considerations.

Following interoperable data conventions ensures that the system can be extended more easily in future versions, for example through the addition of new monitoring modules, analytics services, or dashboard functions. It also supports portability and integration with external systems if the project evolves beyond its current prototype scope.

6.2.5 Modular Software Engineering Principles

From a software engineering perspective, Moo-tion is developed according to widely accepted principles of modularity, separation of concerns, and maintainability. The system architecture separates major functions such as video acquisition, image processing, backend logic, and user interface services into logically distinct layers. This design approach reflects standard engineering practice for reducing complexity and improving system robustness.

A modular design also facilitates testing, debugging, and future improvement. Since the project may need to operate under different farm conditions and may later incorporate additional functionalities, adherence to structured software development principles is essential for long-term sustainability.

6.2.6 Reliability, Maintainability, and Documentation Practices

Another important standard-related aspect of the project is the adoption of general engineering practices for reliability and maintainability. Even though Moo-tion is developed within an academic project framework, the implementation is expected to follow disciplined coding, documentation, and system organization practices. Clear interface definitions, readable code structure, and documented component responsibilities contribute to the reproducibility and maintainability of the system.

Such practices are especially important for interdisciplinary projects, where future contributors may need to understand, modify, or extend the system. Therefore, documentation and maintainable implementation are treated not merely as optional development habits, but as essential engineering standards supporting the practical usability of the project.

6.2.7 Privacy and Responsible System Operation

Because Moo-tion processes video-based observations in real farm environments, the project also takes into account the need for responsible handling of visual data and system access. Although the primary focus of the project is technical functionality, the use of camera-based monitoring in private agricultural environments requires attention to controlled access, restricted visibility of system outputs, and appropriate handling of operational data.

For this reason, the system is designed with an awareness of basic privacy-preserving and access-limited operation principles. This does not imply compliance with a formal legal certification framework within the scope of the

project, but it reflects a responsible engineering approach to the deployment of monitoring technologies in real environments.

7. Teamwork Details

7.1. Contribution

The Moo-tion project was carried out through a collaborative distribution of responsibilities among team members, with each member contributing to a specific technical or operational aspect of the system. Due to the interdisciplinary nature of the project, the workload was divided across frontend development, backend infrastructure, artificial intelligence, computer vision, camera integration, system coordination, and communication with external stakeholders. This division enabled the team to develop the project in a structured and efficient manner while maintaining close interaction between software and field-related tasks.

- Emir Eroğlu was primarily responsible for frontend development and the artificial intelligence model used in the system. In addition to implementing the user-facing software components, he contributed to the preparation and refinement of the model pipeline, ensuring that the outputs of the intelligent monitoring system could be meaningfully integrated into the interface. His work also supported the presentation of system results in a clear and accessible way for end users.
- Canberk Özba focused on computer vision tasks related to cow tracking and recognition. His contribution involved the development of image processing and analysis mechanisms required for detecting animals, following their motion, and supporting identification within the camera view. He also contributed to the evaluation of visual outputs and the improvement of tracking consistency under farm conditions.
- Muhammed Enes Şahin was responsible for the camera stream acquisition processes and the integration of visual input into the system pipeline. His work involved accessing image streams from existing farm cameras, handling stream-related technical issues, and ensuring that camera data could be used reliably by the vision modules. In addition to this role, he contributed to the coordination of system-level integration between camera infrastructure and the rest of the software architecture.
- Ahmet Burak Kılıçkaya was responsible for backend development and communication with farms. His contribution included the design and implementation of backend-side functionalities necessary for data flow, service operation, and system coordination. He also handled

communication with farm stakeholders, which played an important role in obtaining access to practical deployment conditions, understanding field requirements, and evaluating the applicability of the solution in real environments.

- Mert Özkaya contributed to the computer vision subsystem, particularly in the areas of cow tracking and recognition. His work supported the implementation and refinement of image-based monitoring functions and complemented the efforts of other team members working on visual analysis. In addition, his contribution helped strengthen the technical depth of the tracking pipeline and supported testing under different visual conditions.

7.2. Collaborative and Inclusive Environment

The Moo-tion project was carried out in a collaborative and open team environment supported by regular communication and shared decision-making. Team members maintained communication through WhatsApp, face-to-face meetings, and Google Meet sessions, depending on availability and project needs. This combination of communication channels allowed the team to remain connected both during routine development periods and during more intensive stages of the project.

In general, the team held short progress meetings approximately once every two weeks in order to evaluate the current status of the project, review ongoing tasks, and identify immediate needs. During periods close to important deadlines, meetings were held more frequently in order to accelerate coordination and ensure timely completion of deliverables. This flexible meeting structure helped the team adapt its working rhythm according to project intensity.

Project decisions were made collectively. Rather than relying on a rigid top-down structure, the team discussed alternatives together during meetings and moved forward based on shared agreement. This approach ensured that each member remained informed about the overall progress of the project and that important technical or organizational choices were made with group awareness.

GitHub was used as a core collaboration tool during development. It supported code sharing, version tracking, and coordination between different parts of the software system. In addition to formal task progress, team members also supported one another through idea exchange, joint problem solving, collaborative research, and short, fast discussions whenever a technical issue

required immediate attention. This created an effective working dynamic in which communication was not limited only to scheduled meetings.

The team environment was also inclusive in terms of participation. Each member was able to express ideas comfortably, and the team developed a sincere and supportive internal culture throughout the project. This open atmosphere encouraged discussion and made it easier to bring forward alternative opinions or technical suggestions without hesitation.

When disagreements occurred, the team resolved them through voting. This provided a simple and fair mechanism for moving forward while preserving group balance. In addition, a respectful working environment was supported from the beginning by forming the team carefully and choosing compatible teammates who could work together constructively. As a result, collaboration within the project remained productive, respectful, and solution-oriented.

8. Conclusion

Moo-tion has been developed as a computer vision based livestock monitoring system designed to operate on existing farm camera infrastructure. The project addresses the need for a practical, non-invasive, and cost-effective monitoring solution for dairy farms by combining video stream acquisition, image processing, backend services, and user-oriented software components within a unified system architecture. By building on already available IP cameras, the project aims to reduce additional hardware requirements while increasing the accessibility of intelligent monitoring technologies for real farm environments.

Throughout the design process, the project was shaped not only by technical objectives but also by the practical realities of agricultural deployment. As a result, Moo-tion was structured as a modular and scalable system that can support monitoring, analysis, and communication functions within a single framework. The integration of camera-based observation with software-driven analysis demonstrates the feasibility of using existing visual infrastructure for livestock-focused monitoring tasks.

The project also highlighted the importance of designing with real-world constraints in mind. Issues related to camera access, stream quality, viewing angles, lighting conditions, and network reliability showed that successful deployment depends not only on algorithmic performance but also on environmental and infrastructural compatibility. Despite these challenges, the proposed system establishes a solid foundation for intelligent and affordable farm monitoring.

Overall, Moo-tion represents an interdisciplinary engineering effort that combines computer vision, software development, and practical field considerations into a coherent monitoring solution. It provides a meaningful basis for future development in digital livestock technologies, particularly for farms seeking low-cost and adaptable alternatives to sensor-heavy or high-cost monitoring systems.

As future work, the system can be improved through testing on a larger number of farms, expanding robustness under low-light and low-quality imaging conditions, and enhancing multi-camera monitoring capabilities. Further refinement of visual analysis modules may also improve system performance in more complex and variable barn environments.

9. References